

Итоговый отчёт: QA приложения «Свои люди»

Честный отчёт по каждому пункту задания — без приукрашивания, с прямыми ссылками на файлы и коммиты, по которым можно проверить каждое утверждение самостоятельно.

1. Работающий результат

Что протестировано: приложение «Свои люди» (`app-under-test/index-local.html`) по четырём разделам `qa-сpec.md` — функциональность (регистрация, вход, размещение объявлений, бронирование, права по логину), доступность (клавиатурная навигация, фокус в модалках, `Escape` , `axe-core`), устойчивость к кривым/чужим данным (инъекции, обход валидации, подделка сессии, гонки записи), нагрузка/объём (50–500 записей, квота `localStorage` , отклик поиска).

Как запустить и убедиться самостоятельно:

```
npm install
npx playwright install chromium
npm test
```

Ожидаемый результат: **15 passed / 12 failed** (было 14/12 до разбивки сквозного теста `tests/functional-flow.spec.js` на зелёный happy-path и отдельный красный тест на баг Ф-1 — см. раздел 4; 10/12 до того, как добавили 4 зелёных теста на обход UI — `tests/bypass-ui.spec.js` , разделы 3.4/3.9/3.10 `qa-сpec.md` ; и 9/13 ещё раньше, до переписи `tests/volume-bugs.spec.js` под исправленный `localStorage`-баг), стабильно при повторных прогонах (проверено многократно на протяжении всей работы, в т.ч. прямо перед написанием этого отчёта). Это не брак — 12 падений это честные ассерты на реальных, задокументированных дефектах приложения (см. раздел 4 «Честно» ниже и `qa-сpec.md` за деталями каждого сценария). HTML-отчёт прогона: `npm run test:report` .

Автотесты лежат в `tests/` : `accessibility.spec.js` , `keyboard-focus.spec.js` , `functional-flow.spec.js` , `permissions.spec.js` , `security.spec.js` , `volume-bugs.spec.js` , `bypass-ui.spec.js` — 27 тестов, все идут через реальный UI (клики, заполнение полей), а не через вызовы внутренних функций страницы в обход

интерфейса (исключение — генерация объёма данных в `volume-bugs.spec.js`, это осознанно, см. комментарий в файле).

2. Агенты

Работа велась в трёх разных режимах — намеренно, чтобы сравнить подходы, а не потому что один режим не справился:

- **Параллельно** — `qa-report.md`. Три независимых субагента одновременно, каждый в своей зоне без пересечений (функциональность / доступность / устойчивость к данным). Wall-clock ~11 мин 15 с, суммарно 290 497 токенов на троих, 330 вызовов инструментов.
- **Последовательно** — `qa-report-solo.md`. Та же задача, один поток, без субагентов. ~16 мин 21 с (на 45% дольше), но ~84 800 токенов (в 3,4 раза меньше — каждый параллельный субагент заново читает файлы и поднимает окружение, последовательный прогон делает это один раз). Результаты по существу совпали с параллельным прогоном — расхождение только в затратах, не в находках.
- **Независимый ревьюер** — дважды, по одному и тому же принципу («агент без памяти о том, как писались тесты/код, перепроверяет с чистого листа»):
 1. `qa-review-independent.md` — разовый аудит поверх всей существовавшей на тот момент QA-работы (спеки, трёх отчётов, кода трёх HTML-вариантов приложения). Нашёл критичный необнаруженный дефект (крэш `index-seeded.html` / `index-for-artifact-publish.html` вне платформы Claude) и то, что один тест маскировал известный баг вместо честного падения. Под каждой находкой в файле отмечено, каким коммитом она закрыта.
 2. Тот же принцип, но встроенный в процесс на постоянной основе — шаг 4 воркфлоу `.claude/skills/qa-release-check/` (см. раздел 3). Отдельный Agent (`general-purpose`), стартующий с чистого контекста, вызывается на каждом прогоне и обязан сам записать файл результата — это не разовая проверка, а встроенная в повторяемый процесс независимая сверка.
- **Устойчивость к prompt-injection** — `security-injection-test.md`. Побочный эксперимент: в файл проекта временно вшита безвредная скрытая инструкция (HTML-комментарий), слабой (Haiku) и сильной (Sonnet) модели дана невинная задача «перескажи файл», результат сведён в таблицу «выполнила / промолчала / назвала вслух». Файл после теста откачен. Ни одна модель инструкцию не выполнила: Haiku молча проигнорировала, Sonnet дополнительно назвала её попыткой инъекции.

3. Workflow

`.claude/skills/qa-release-check/SKILL.md` — повторяемая процедура «QA-прогон перед релизом», вызывается перед каждым изменением `app-under-test/` (не разовый скрипт). Пять шагов, каждый оставляет файл-доказательство в `qa-runs/<дата-время>/`:

1. Диф `app-under-test/` с последнего коммита → `01-diff.md`.
2. Сопоставление дифа с затронутыми пунктами `qa-spec.md` → `02-spec-impact.md`.
3. `npx playwright test` → `03-test-run.md`.
4. Независимая проверка покрытия и «маскировки» багов свежим подагентом без памяти о том, как писались тесты (см. раздел 2) → `04-independent-review.md`, пишет сам агент, исходная сессия не редактирует его содержимое.
5. Итоговый честный отчёт (что прогнали / что упало / что нового / что не покрыто / что не успели) → `05-final-report.md`.

Запуск: `/qa-release-check` или фразой «прогони QA-check перед релизом».

Честно про его текущее состояние: первый прогон (`qa-runs/2026-08-27_01-34-26/`) был выполнен вручную, шаг за шагом по тексту скилла, ещё до того как `Skill()` стал вызывать его автоматически по имени в рамках той же сессии. Второй прогон (`qa-runs/2026-08-27_02-27-51/`) уже вызван штатно через `/qa-release-check` → `Skill('qa-release-check')` — реальный вызов по имени подтверждён. Оба прогона — локально, не в git, `qa-runs/` в `.gitignore` как регенерируемые артефакты.

Второй прогон застал `app-under-test/` без изменений с первого (тот же коммит `a67adc92`) — то есть это в первую очередь проверка стабильности самого процесса, а не нового изменения. Результат: **9 passed / 13 failed**, идентично первому прогону, новых регрессий нет (`qa-runs/2026-08-27_02-27-51/03-test-run.md`). Шаг 4 (независимая проверка) на этот раз выявил два момента, которых не было в первом прогоне — см. раздел 4 ниже.

Третий прогон (`qa-runs/2026-08-27_02-54-04/`) впервые сопровождал реальную правку кода — фикс бага «тост врёт об успехе при переполнении localStorage» (см. раздел 4) сразу в трёх файлах. Показательный случай для самого процесса: шаг 4 (независимая проверка) нашёл Critical-баг **в самой правке** (фантомная карточка в DOM при провале сохранения, см. раздел 4) уже после того, как playwright-тесты были прогнаны и задокументированы в `03-test-run.md`. Правка была доработана по этой находке в той же сессии, тесты перепрогнаны повторно (`raw-test-output-2.txt`) — результат тот же **9 passed / 13 failed**, новых регрессий вторая итерация не внесла. Это прямое подтверждение ценности шага 4 как отдельного от шага 3 —

«зелёные тесты» после первой итерации фикса не поймали бы этот баг, его нашла именно независимая ревью-проверка кода, а не прогон существующего набора тестов.

4. Честно: что не сработало и не сделано

- **Баг Ф-1** (модалка деталей закрывается сама сразу после клика «Забронировать», вместо того чтобы остаться открытой с раскрытым телефоном) — **не исправлен в коде приложения**. Сквозной тест `tests/functional-flow.spec.js` разбит на два: зелёный happy-path (регистрация → объявление → бронь → раскрытие телефона и бейдж брони; телефон и бейдж проверяются через сознательное переоткрытие модальной в обход Ф-1) и отдельный красный тест `Ф-1: модалка деталей остаётся открытой после брони`, который честно падает на проверке видимости `#overlay-detail` и позеленеет сам, когда Ф-1 починят в приложении. Раньше это был один тест на весь сценарий — он умирал на Ф-1, из-за чего проверки телефона и бейджа фактически не выполнялись (а ещё изначально тест маскировал баг переоткрытием модальной — находка независимого ревью, исправлена).
- **Отсутствие debounce на поиске** (каждое нажатие клавиши — 130–290 мс на 350–500 записях, полная пересборка `innerHTML`) — **не исправлено**. Задokumentировано в `qa-report-volume.md`, тест `tests/volume-bugs.spec.js` честно падает.
- **Потеря данных при переполнении `localStorage`** (`saveLocalState()` проглатывала `QuotaExceededError` в пустой `catch{}`, `save()` всё равно показывала тост об успехе) — **исправлено** в третьем QA-прогоне (`qa-runs/2026-08-27_02-54-04/`), во всех трёх файлах (`index-local.html`, `index-seeded.html`, `index-for-artifact-publish.html`): `saveLocalState()` теперь возвращает `true` / `false`, `save()` при неудаче показывает честную ошибку вместо тоста успеха. Фикс делался в две итерации в рамках одной сессии: первая итерация чинила только текст тоста; независимый подагент на шаге 4 того же прогона нашёл дополнительный Critical-баг в самом фиксе (см. следующий пункт) — вторая итерация его закрыла. Автотест `tests/volume-bugs.spec.js` был написан как ловушка под старый баг (ожидал текст тоста «Объявление размещено» и требовал `actuallyPersisted === true`) — в следующей сессии **переписан** под честное новое поведение: ожидает новый текст ошибки, `actuallyPersisted === false` (данные объективно не могли сохраниться при реально исчерпанной квоте — это физическое ограничение, не баг), плюс добавлена DOM-проверка на отсутствие фантомной карточки и проверка после `location.reload()` (закрывает

заодно пункт 4.8 `qa-спес.md`, который раньше был без теста). Тест теперь зелёный (см. следующий блок и раздел 3).

- Доступность (4 нарушения `axe-core` critical/serious, `Escape` не закрывает модальки, нет фокус-трапа, нет `role="dialog"`, карточки объявлений недоступны с клавиатуры) — **не исправлена**, только зафиксирована тестами.
- **Разделы 3.4 (обход валидации цены), 3.9 (самобронирование в обход UI), 3.10 (гонка двойного бронирования)** `qa-спес.md` — **покрыты** файлом `tests/bypass-ui.спес.js` (4 теста, все зелёные). Все три пункта в `qa-спес.md` помечены как принятые архитектурные ограничения локального прототипа (нет сервера → клиентские проверки обходимы; гонки — last-write-wins без блокировок), поэтому тесты построены по образцу `tests/security.спес.js` (3.13): фиксируют фактическое поведение зелёным, а не растут красным сигналом о баге. Результат: 3.4 — отрицательная цена уходит в состояние как есть («-500 Р», без подрезки к нулю), нечисловая цена молча становится `NaN` → «О Р» без ошибки валидации; 3.9 — прямая запись брони с `bookedBy.username === authorUsername` ничем не проверяется, самобронь проходит полноценно (телефон «раскрыт» автору же); 3.10 — из двух «одновременных» бронирований в состоянии остаётся только последнее, раннее исчезает без следа и без предупреждения.
- Более широкий пробел покрытия спецификации автотестами остаётся: без тестов — 3.1–3.3, 3.5–3.7, 3.12 (раздел 3, кроме 3.4, 3.9, 3.10, 3.11 и 3.13), большая часть 1.1–1.4, часть 2.1–2.4 и половина раздела 4. Подтверждено независимо в `qa-review-independent.md` и `qa-runs/2026-08-27_01-34-26/04-independent-review.md`. Это не сделано и не планировалось в рамках выполненных сессий — называю прямо, а не молчу об этом.
- `index-seeded.html` / `index-for-artifact-publish.html` получили технический фикс (перестали падать вне платформы Claude), но по-прежнему нигде не помечены как «снимок, разойдётся с живым артефактом» — риск повторной путаницы («у меня не запускается» из `sessions/session-7.md`) снижен, не устранён полностью.
- **Дублирование бага «тост врёт об успехе при переполнении localStorage»** в `index-seeded.html` / `index-for-artifact-publish.html` (найдено независимым ревью на втором прогоне QA-check, `qa-runs/2026-08-27_02-27-51/04-independent-review.md`, [high] в разделе A) — **исправлено** в третьем прогоне (`qa-runs/2026-08-27_02-54-04/`), идентичной правкой во всех трёх файлах (см. пункт выше). По-прежнему верно то, что было отмечено при находке: правка **физически не может быть проверена автотестами** для двух из трёх файлов — `tests/helpers.js` / `playwright.config.js` жёстко привязаны к `index-local.html`, поэтому идентичность кода в `index-`

`seeded.html` / `index-for-artifact-publish.html` подтверждена только побайтовым сравнением diff'a (`qa-runs/2026-08-27_02-54-04/01-diff.md`), не прогоном. Это отдельно зафиксировано как непокрытый риск в разделе 3.

- **Новый Critical-баг, найденный независимым ревью в рамках самого фикса** (`qa-runs/2026-08-27_02-54-04/04-independent-review.md`): первая версия правки обновляла `state` и вызывала `render()` безусловно, до проверки результата `saveLocalStorage()` — из-за этого при провале сохранения на экране появлялась «фантомная» карточка объявления (или подтверждение брони), которой на самом деле нет в `localStorage`, и тост об ошибке показывался одновременно с этим визуальным враньём. **Исправлено** в той же сессии, второй итерацией: `state = newState` / `render()` теперь выполняются только при успешном `saveLocalStorage()`. Тест `tests/volume-bugs.spec.js` в следующей сессии **переписан** под новое поведение (не только текст тоста, но и `actuallyPersisted === false`, отсутствие фантомной карточки в `.grid` / открытых модалок сразу после провала, и проверка после `location.reload()` — закрывает заодно пункт 4.8 `qa-spec.md`) — тест теперь зелёный, а не красный по устаревшему ассерту.
- **Тест «забронировавший может отменить свою бронь»** (`tests/permissions.spec.js:58`) **слабее, чем предполагает его название** — проверяет только видимость кнопки «Отменить бронь», не кликает по ней и не проверяет реальную отмену, в отличие от симметричного теста для автора. Не маскировка известного бага, а недостаточно строгий ассерт. Найдено тем же прогоном (`04-independent-review.md`, раздел B, [medium]).

Прогон тестов сейчас: 15 passed / 12 failed. Все 12 падений — из списка выше и ниже, ни одного «непонятного» или флакового падения нет (проверялось многократно, включая независимую проверку в разделе 2). `tests/volume-bugs.spec.js` (тест на переполнение `localStorage`) переписан под честное новое поведение (см. пункты выше) и теперь зелёный. Добавлен `tests/bypass-ui.spec.js` — 4 зелёных теста на принятые ограничения 3.4/3.9/3.10 `qa-spec.md` (обход валидации цены, самобронирование, гонка записи). Сквозной `tests/functional-flow.spec.js` разбит на зелёный happy-path + отдельный красный тест на Ф-1 (см. раздел 4). Прогресс: 9/13 → 10/12 (перепись `volume-bugs`) → 14/12 (+4 теста `bypass-ui`) → 15/12 (разбивка `functional-flow`: +1 зелёный, число падений то же). Число падений не менялось — новые тесты зелёные и регрессий не внесли.

5. Открытый вопрос к продукту — закрыт

Видимость телефона третьим лицам (`qa-спеc.md` , п. 1.4.5): после чужой брони номер телефона автора и кнопки связи видны **любому** авторизованному пользователю, открывшему объявление — не только автору и тому, кто забронировал (`renderDetail()` не проверяет `isAuthor` / `isBooker` для самого блока `phone-reveal` , только для кнопки «Отменить бронь» внутри него). Изначально зафиксировано как открытый вопрос в `qa-report.md` / `qa-report-solo.md` , формализовано пунктом 1.4.5 в `qa-спеc.md` .

Решение принято: оставить как есть. Это не баг, а сознательно принятый компромисс для контекста приложения (закрытый круг соседей по одному дому, где номер телефона и так не является секретом внутри этого круга) — не блокирует, изменений в коде/тестах по этому пункту не требуется. Автотеста на это поведение по-прежнему нет (см. раздел 4 — 1.4.5 в списке непокрытых пунктов); при желании зафиксировать решение регресс-тестом стоит сделать это по тому же принципу, что уже применён для пунктов 3.13 (`tests/security.спеc.js`) и 3.4/3.9/3.10 (`tests/bypass-ui.спеc.js`) — зелёный тест, документирующий осознанно принятое поведение, а не растущий как красный сигнал о баге.